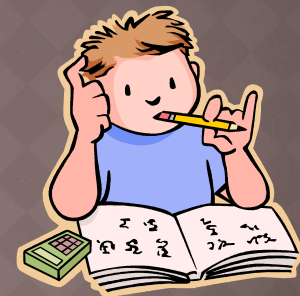




PARADIGMA ORIENTADO A OBJETOS

UML: Lenguaje de Modelado Unificado



CONTENIDO

- ¿Qué es UML?
 - Diagramas UML
- Diagrama de clases
 - Clase, objetos
 - Relaciones entre clases
 - Asociación
 - Dependencia
 - Agregación
 - Composición
 - Herencia
 - Realización
- Diseño OO



¿QUÉ ES UML?



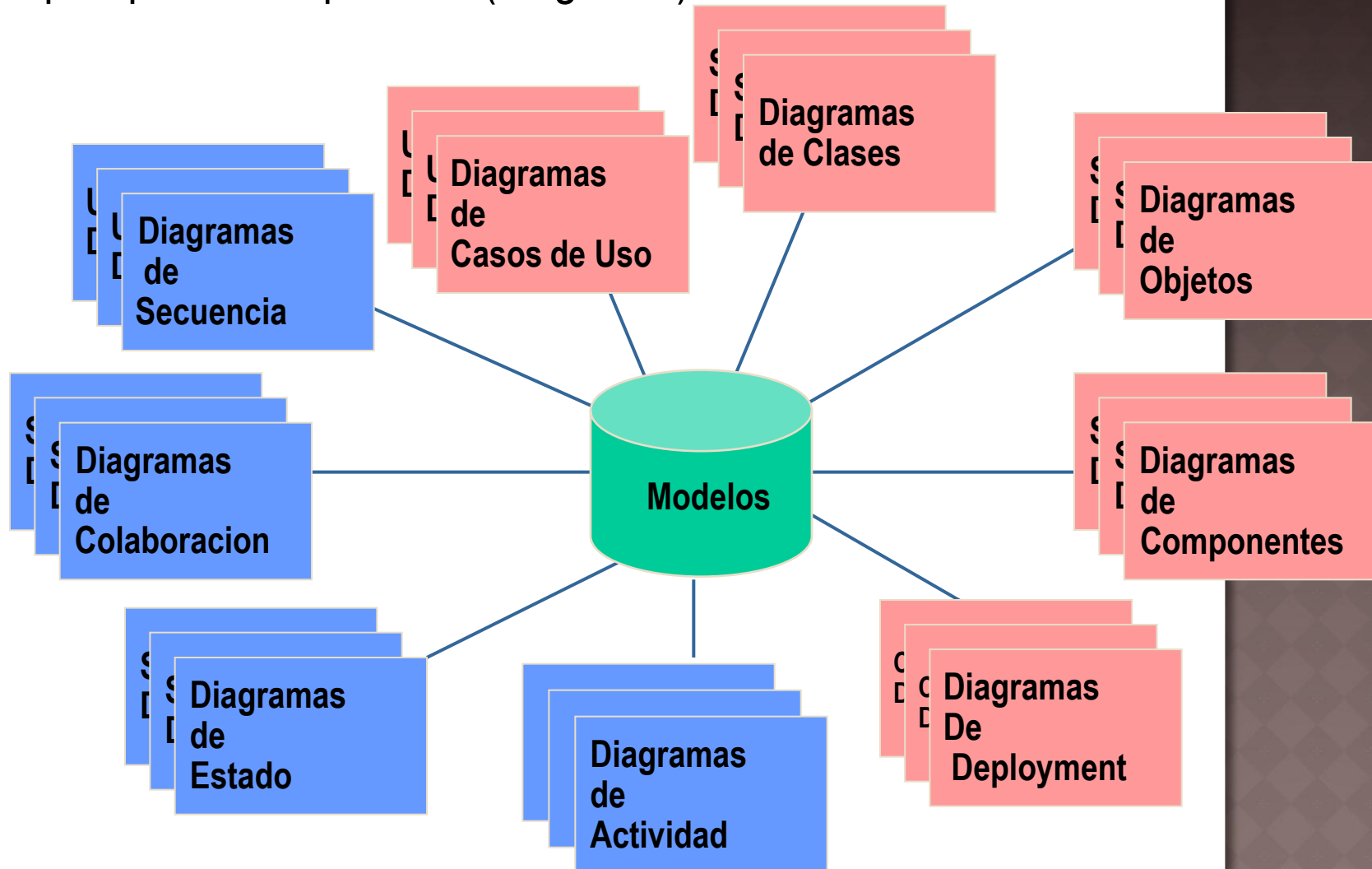
- ◉ UML (Unified Modeling Language) es un lenguaje que permite visualizar, modelar, construir y documentar los elementos que forman un sistema de software usando objetos.
 - Es decir, para realizar un proyecto debemos antes realizar un esquema del mismo.
- ◉ El UML nos permite mediante **diagramas**, plasmar de una forma detallada e inteligible la solución a un problema planteado.

DIAGRAMAS EN UML

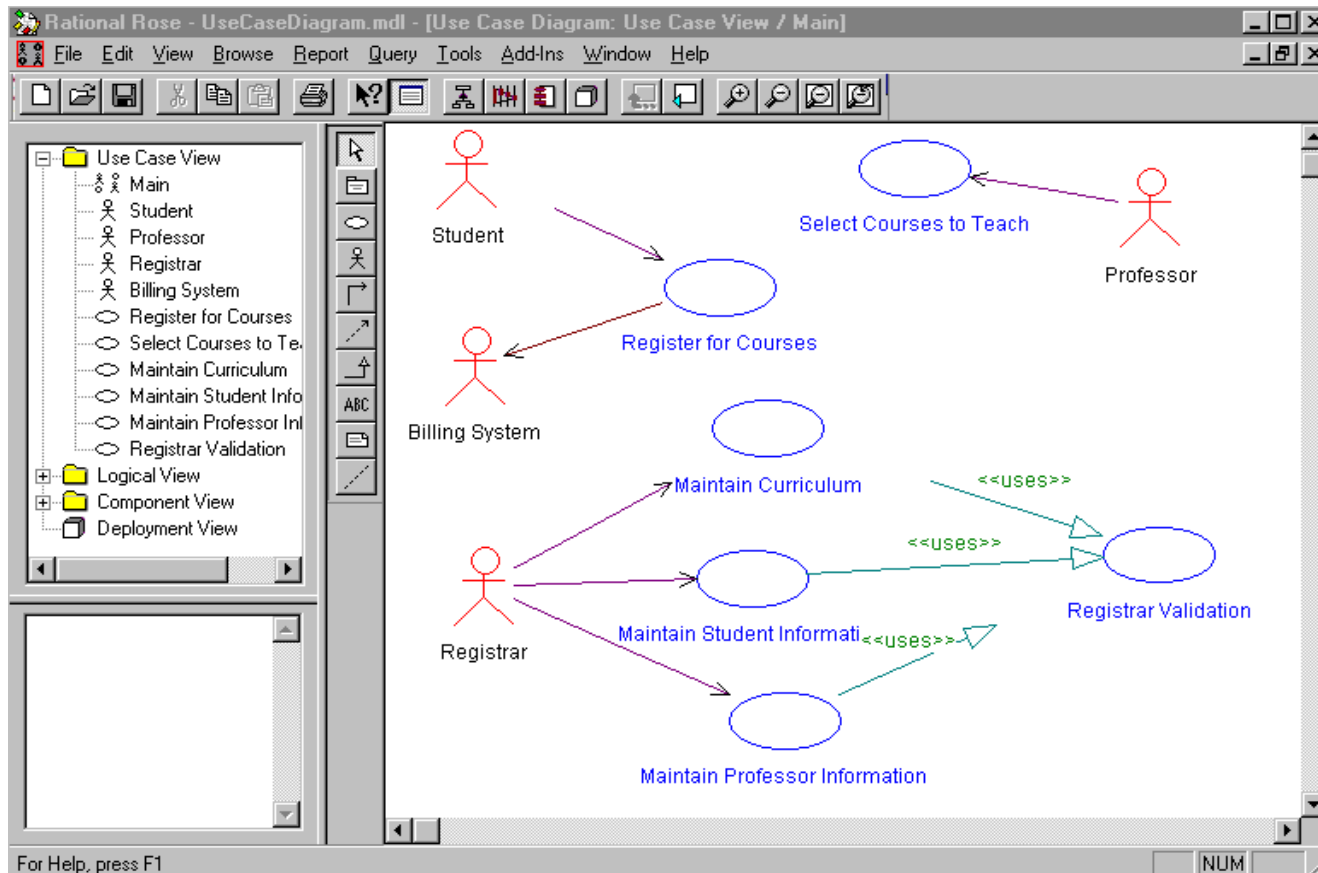
- ◉ UML está compuesto por diversos elementos gráficos que se combinan para conformar diagramas
- ◉ Los diagramas tienen como objetivo presentar diversas perspectivas de un sistema. A esto se le llama **Modelo**.
- ◉ El modelo UML de un sistema es similar a un modelo a escala de un edificio junto con la interpretación del arquitecto del edificio.
- ◉ Tenemos que tener en cuenta que un modelo UML **describe** lo que supuestamente **hará un sistema**, pero **no dice** como **implementar** dicho sistema.

MODELOS Y DIAGRAMAS

- Un modelo es una descripción completa de un sistema desde una perspectiva específica (diagrama)

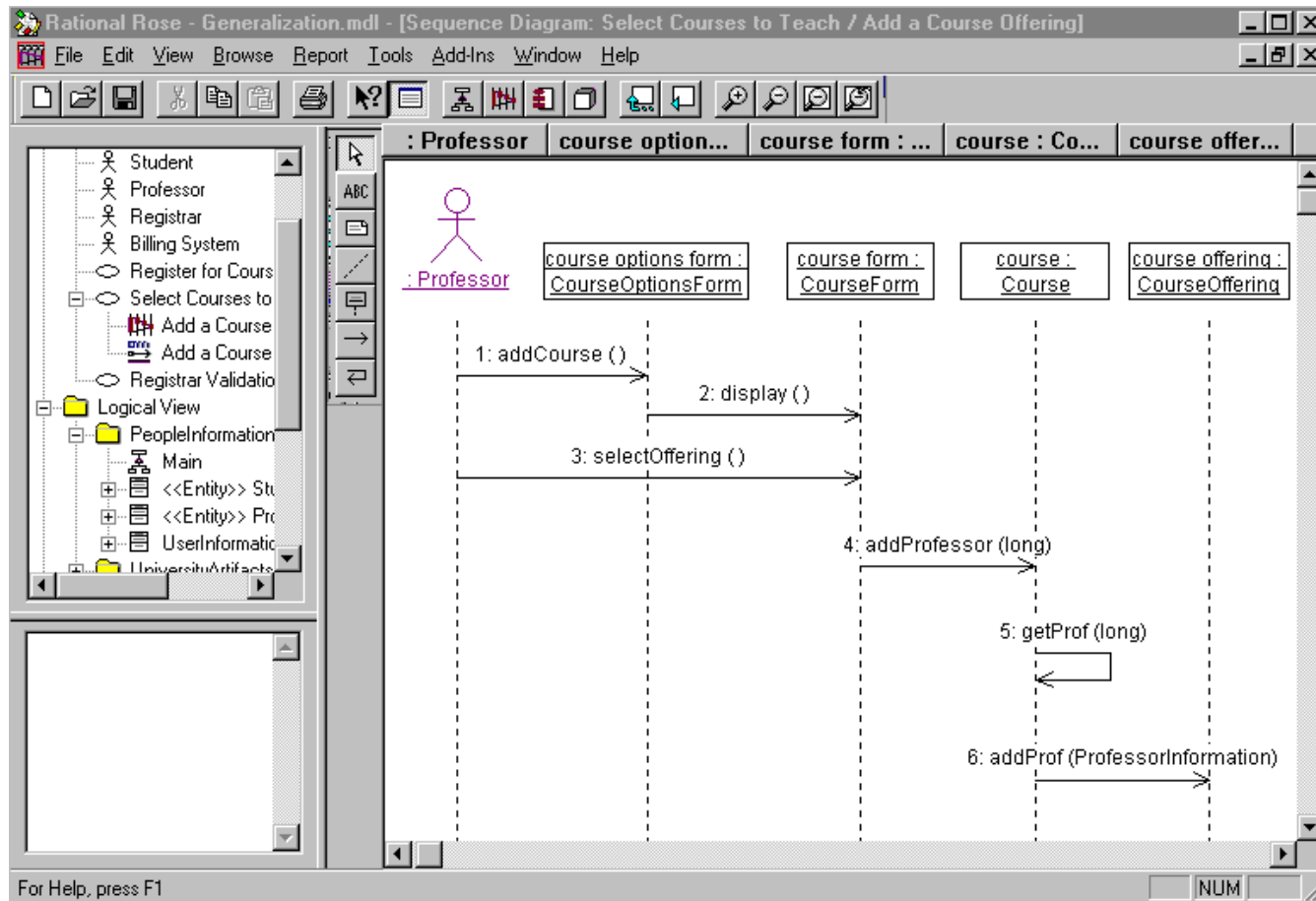


DIAGRAMAS DE CASOS DE USO



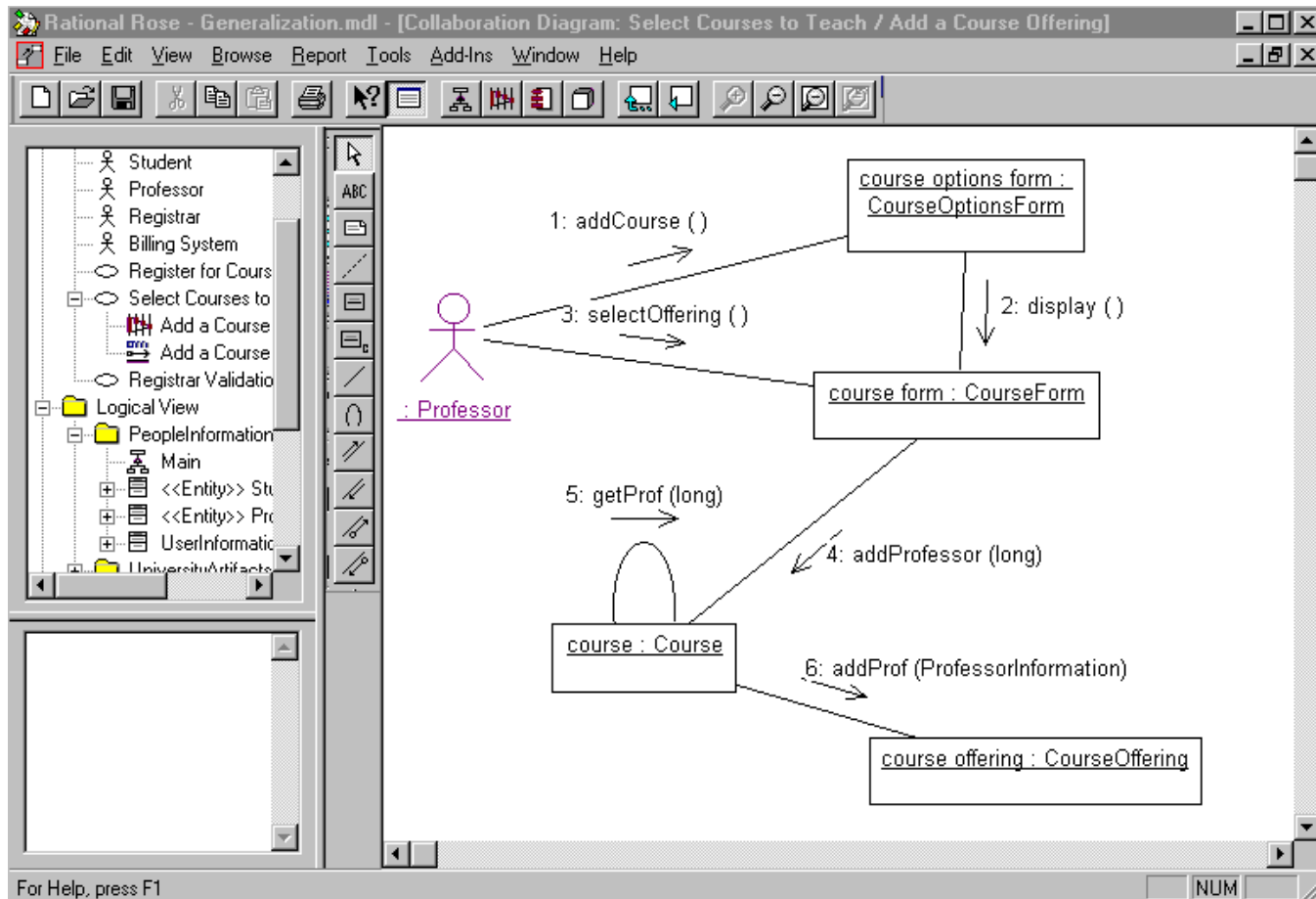
- Se usan los diagramas de Casos de Uso para visualizar la interacción del sistema con el mundo exterior

DIAGRAMAS DE SECUENCIA



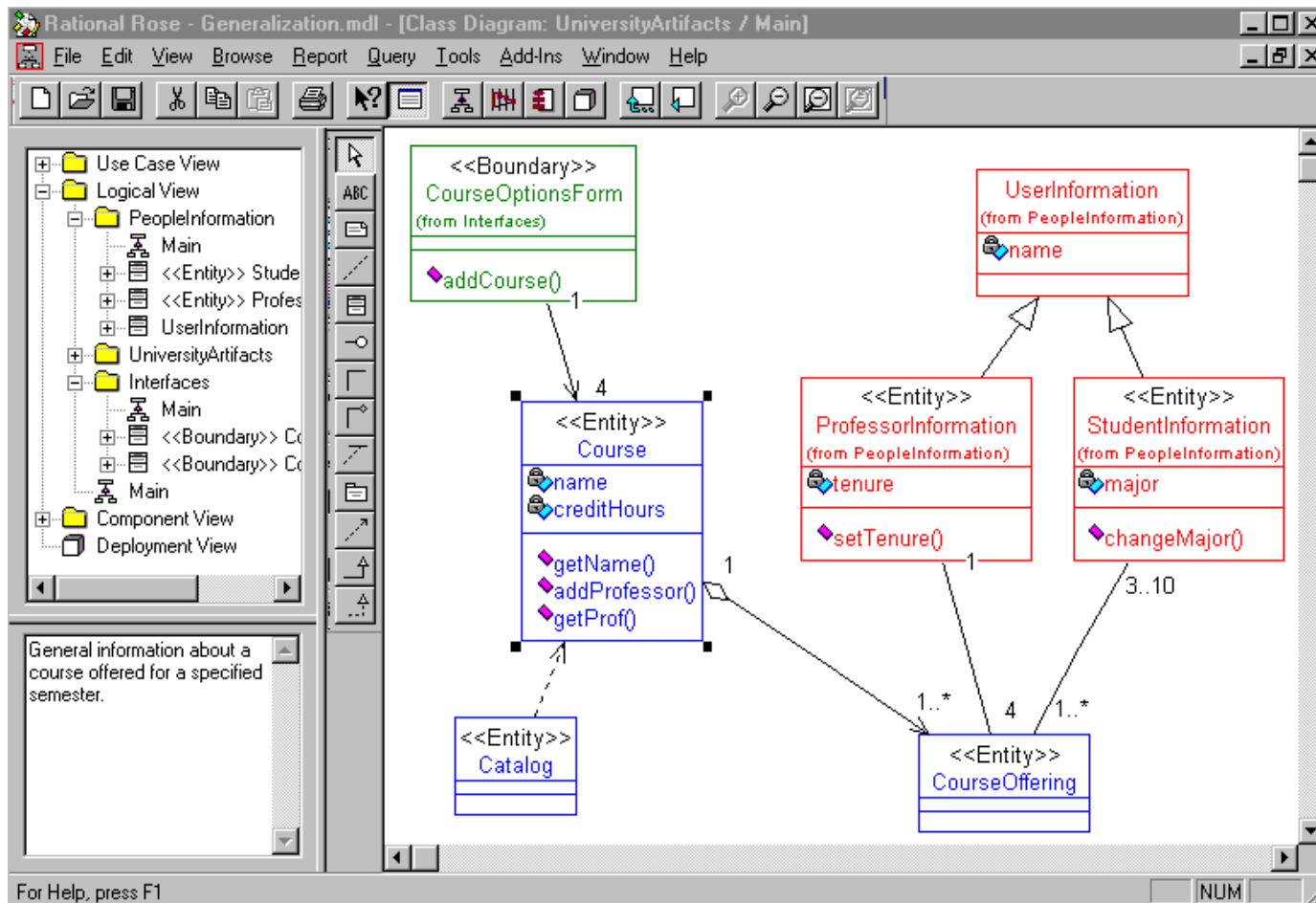
- Los diagramas de secuencia muestran cómo se da paso a paso cierta funcionalidad en el sistema

DIAGRAMAS DE COLABORACIÓN



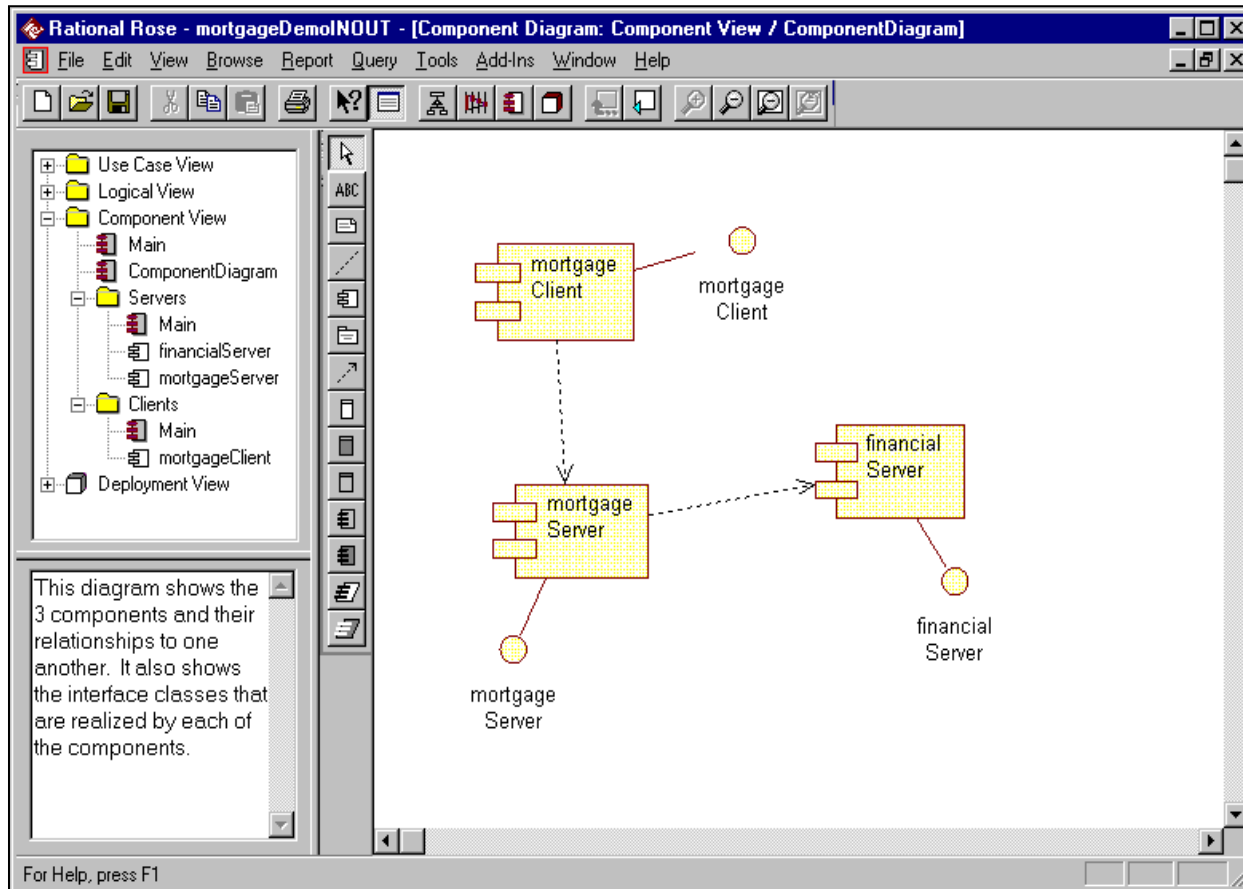
- Los diagramas de colaboración muestran cómo interactúan los objetos para lograr cierta funcionalidad en el sistema

DIAGRAMAS DE CLASE



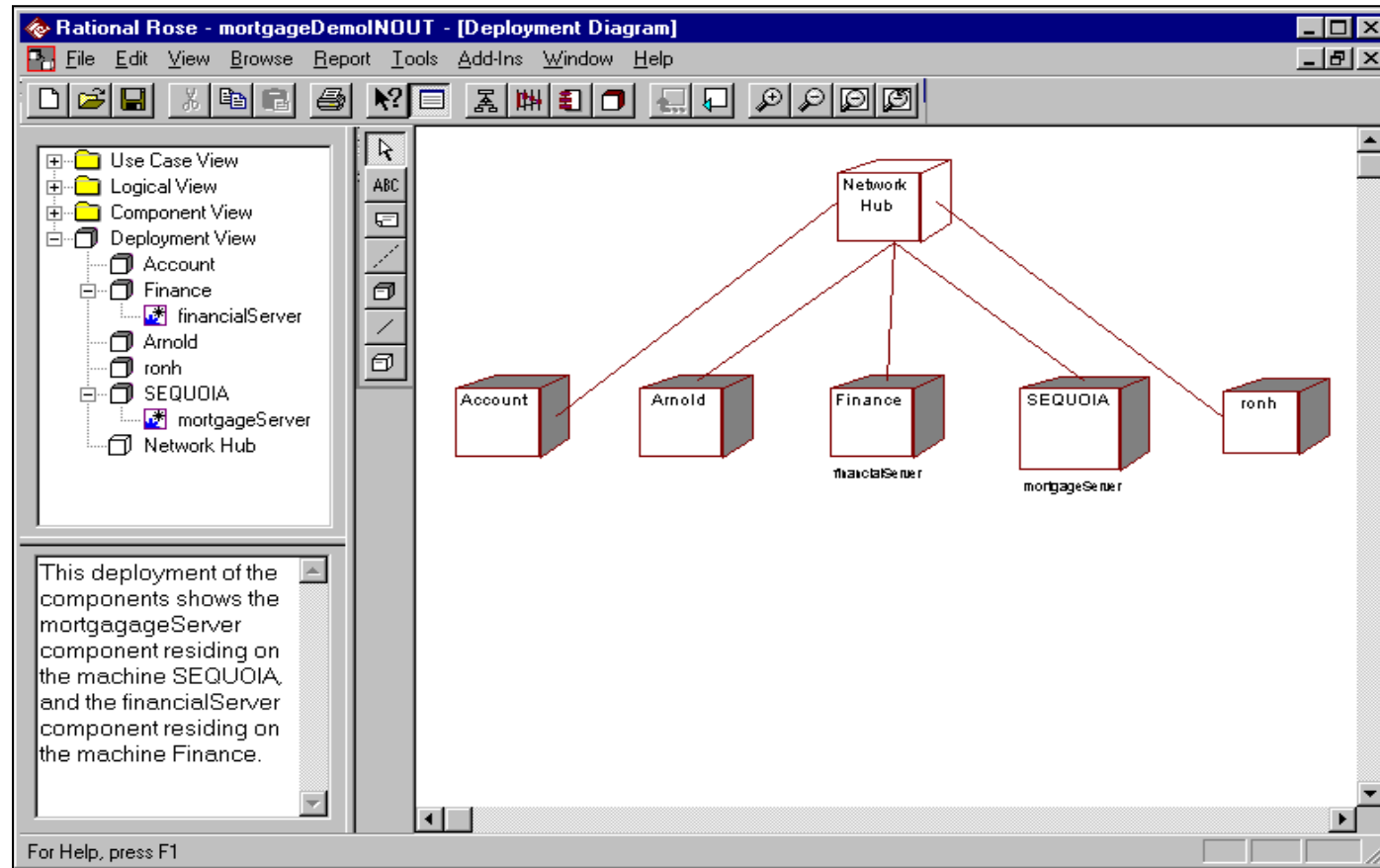
- Los diagramas de clase muestran la estructura del sistema

DIAGRAMAS DE COMPONENTES



- Los diagramas de componentes ilustran la organización y dependencia entre los componentes que definen la arquitectura del sistema

DIAGRAMAS DE DISTRIBUCIÓN



- Los diagramas de deployment permiten visualizar la distribución de los componentes del sistema a través de la empresa

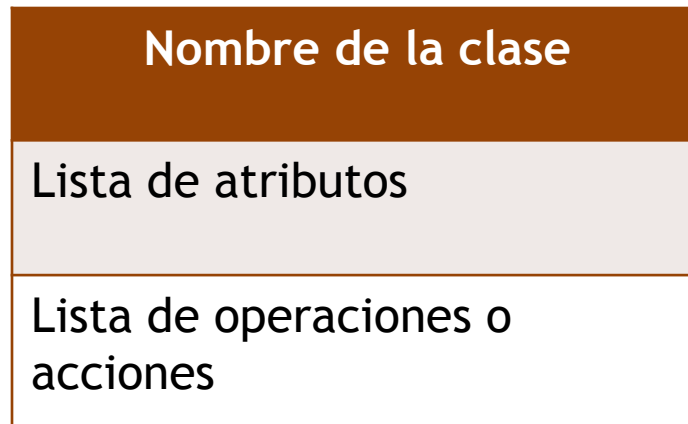
CONTENIDO

- ¿Qué es UML?
 - Diagramas UML
- **Diagrama de clases**
 - **Clase**
 - Relaciones entre clases
 - Asociación
 - Dependencia
 - Agregación
 - Composición
 - Herencia
 - Realización
- **Diseño OO**



DIAGRAMAS DE CLASES

- ◉ En este curso, nos enfocaremos solamente en los **diagrama de clases**, como un medio de diseño de nuestros programas.
- ◉ La forma de representar **una clase** en UML, es mediante una caja con divisiones

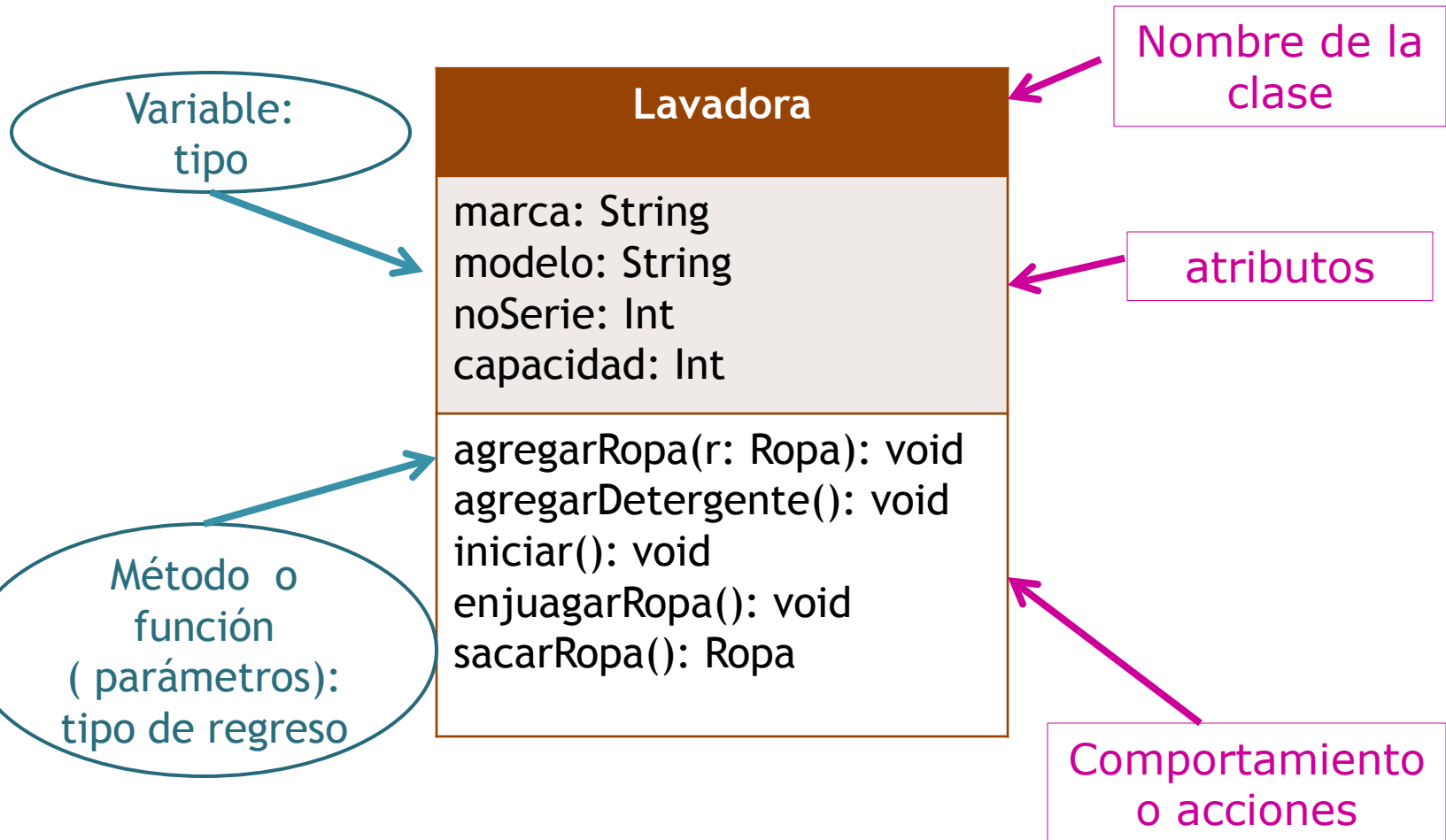


CLASE EN UML

- Recordando algunos conceptos del paradigma orientado a objetos, una **clase** es una categoría o grupo de cosas que tienen atributos y acciones similares.
- Por ejemplo: La clase “lavadora” tiene atributos como, la marca, el modelo, el número de serie y la capacidad. También podemos observar que entre las acciones o comportamientos de esta clase se encuentran: “agregar ropa”, “agregar detergente”, “activarse”, “enjuagar ropa” y “sacar ropa”

Lavadora
marca modelo noSerie capacidad
agregarRopa() agregarDetergente() iniciar() enjuagarRopa() sacarRopa()

ESPECIFICACIÓN DE TIPOS EN UML



ENCAPSULACIÓN EN UML

MÉTODOS DE ACCESO

Símbolo	Indica
-	privado (oculto)
+	público (visible)
#	protegido (semi-oculto)

acciones públicas =
interface!



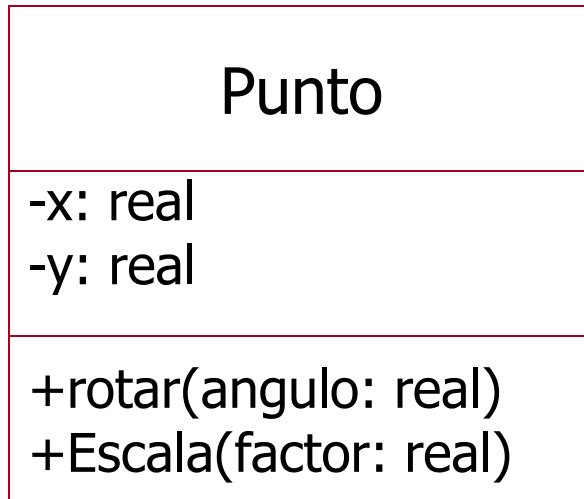
Lavadora

- marca: String
- modelo: String
- noSerie: Int
- capacidad: Int

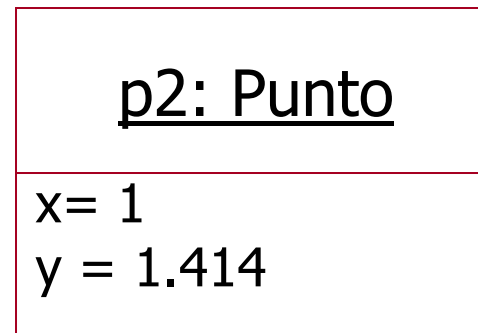
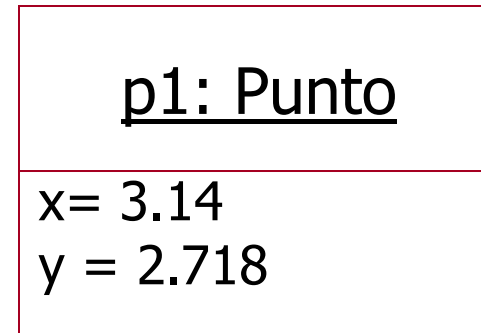
+ agregarRopa(r: Ropa): void
+ agregarDetergente(): void
+ iniciar(): void
+ enjuagarRopa(): void
+ sacarRopa(): Ropa

CLASES Y OBJETOS EN UML

CLASE



Objetos



ALGUNAS CONVENCIONES

- ◉ Nombres de las clases:
 - Inician con Mayúscula
 - Son en singular
 - Se aplica notación camello
- ◉ Nombre de atributos
 - sustantivos
 - Inician con minúscula
 - Se aplica notación camello
- ◉ Nombre de los métodos
 - Verbos (acciones)
 - Inician con minúscula
 - Se aplica notación camello

¿Notación camello?



En la notación camello, cuando se unen varias palabras, cada palabra inicia con mayúscula
Ej: lavarRopaSucia

CONTENIDO

- ¿Qué es UML?
 - Diagramas UML
- Diagrama de clases
 - Clase
 - **Relaciones entre clases**
 - Asociación
 - Dependencia
 - Agregación
 - Composición
 - Herencia
 - Realización
- **Diseño OO**

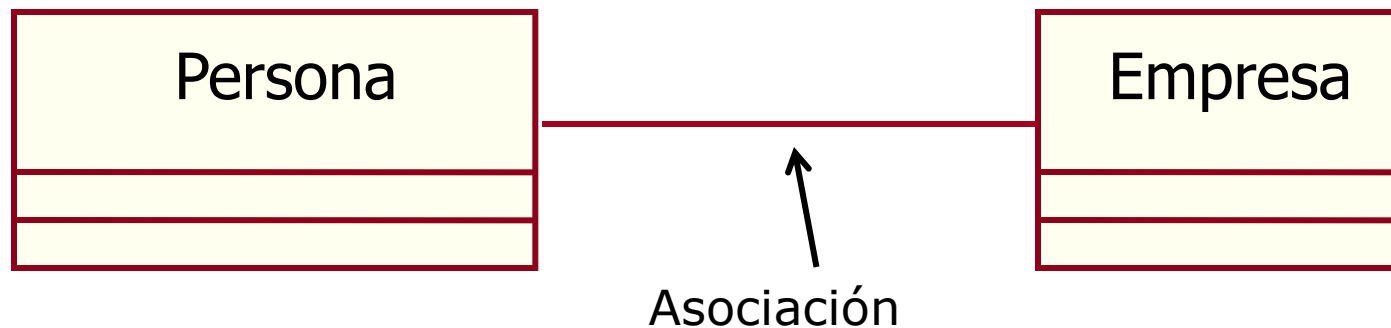


RELACIONES ENTRE CLASES

- ◉ Los problemas del “mundo real”, son más complejos que modelar que una simple clase, más bien, se tiene muchos objetos de distintas clases comunicándose unos con otros. Esto significa, que dichos objetos establecen **relaciones**.
- ◉ Básicamente las relaciones que se establecen son:
 - ❑ Asociación
 - ❑ Dependencia
 - ❑ Agregación
 - ❑ Composición
 - ❑ Herencia
 - ❑ Realización

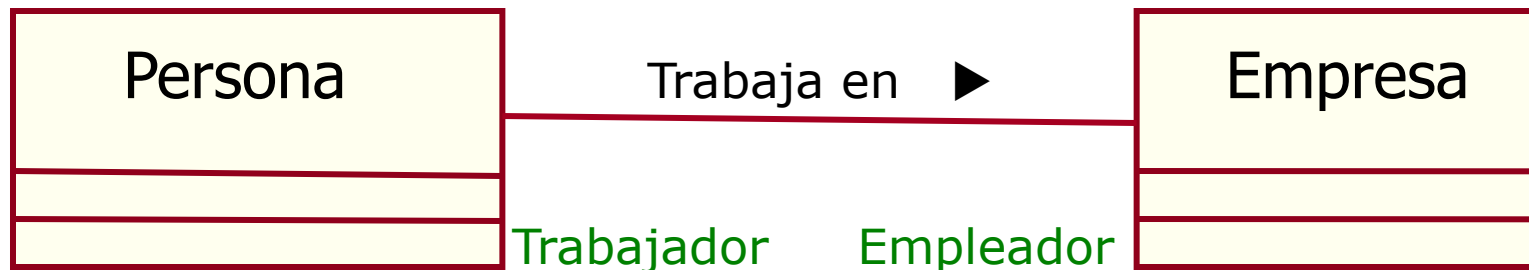
RELACIÓN DE ASOCIACIÓN

- ◉ La relación más simple entre clases es la **asociación**.
- ◉ Indica que dos clases tienen una relación que las asocia según el papel que cada una juega con respecto a otra.
- ◉ Se usa la asociación para relacionar dos objetos que necesitan colaborar enviándose mensajes
- ◉ En un diagrama de clases, la asociación se indica mediante una línea continua que se dirige de un objeto a otro.



RELACIÓN DE ASOCIACIÓN EN UML

- Para aclarar su significado, En la línea se puede anotar el **nombre de la relación** (normalmente un verbo) y con un triángulo se indica la dirección en la que hay que leer dicho nombre.
 - Por default la asociación es bi-direccional.
- Opcionalmente se pueden colocar un nombre de **rol para cada clase**, dicho rol es un nombre alternativo que representa el papel que toma cada clase en la relación



RELACIÓN DE ASOCIACIÓN

- ◉ La Multiplicidad es el número de instancias de una clase relacionadas a UNA instancia de otra clase
- ◉ Para cada asociación existen dos decisiones que tomar en cuanto a multiplicidad; una para cada extremo de la asociación
- ◉ Por ejemplo, en la conexión entre Profesor y Curso
 - Cada instancia de Profesor, puede impartir muchos (cero o más) cursos
 - Para cada instancia de Curso, hay exactamente un Profesor



INDICADORES DE MULTIPLICIDAD

- Cada extremo de la asociación contiene un indicador de multiplicidad
 - Indica el número de objetos que participan en la relación

Sin especificar

Exactamente uno

Muchos (Cero o más,
sin limite)

Uno o más

Cero o uno

Rango Específico

Varios Rangos

1

*

0..*

1..*

0..1

2..4

2,4..6,8

¿QUÉ SIGNIFICA LA MULTIPLICIDAD?

- La multiplicidad responde dos preguntas
 - ¿La asociación es obligatoria o es opcional?
 - ¿Cuál es el mínimo y máximo número de objetos de una clase que se pueden enlazar a un objeto de la otra clase?
- Las decisiones de multiplicidad exponen muchas suposiciones escondidas acerca del problema que se está modelando
 - ¿Se puede ir el maestro de licencia o tomar vacaciones?
 - ¿Puede un curso tener dos maestros?



RELACIÓN DE DEPENDENCIA

- ◉ Esta relación implica **dependencia** entre clases. Se denota mediante una línea punteada con flecha.
 - Siempre es dirigida (La clase A depende de la clase B)
- ◉ La dependencia es causada porque un método en la clase A temporalmente **“usa un”** objeto de la clase B.



RELACIÓN DE DEPENDENCIA

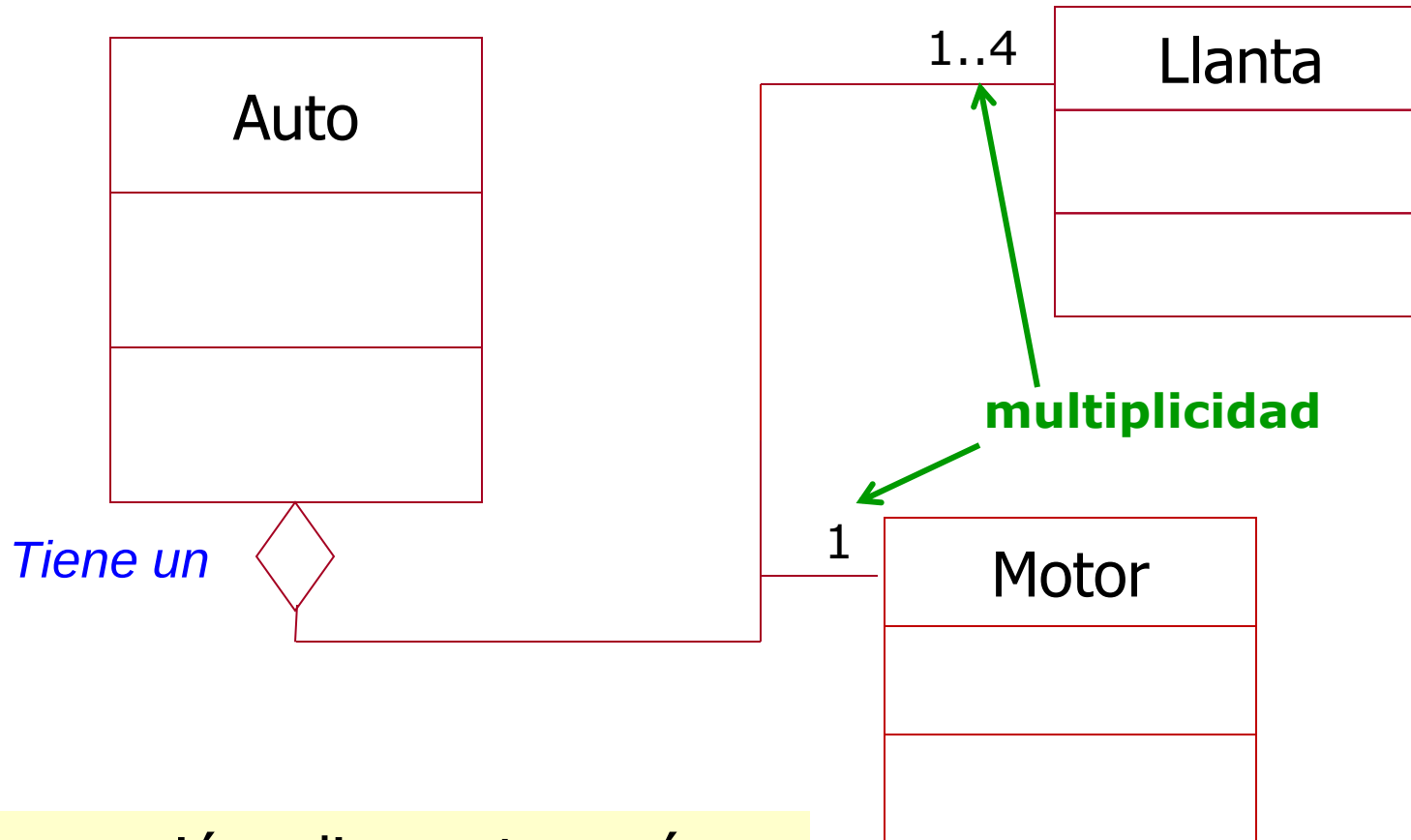
- ◉ Por ejemplo, un *Pedido* **usa una** *Cuenta*, porque un pedido necesita acceder a los objetos de tipo cuenta para comprobar el estado de un crédito
 - La flecha es dirigida hacia la clase “usada”



RELACIÓN DE AGREGACIÓN

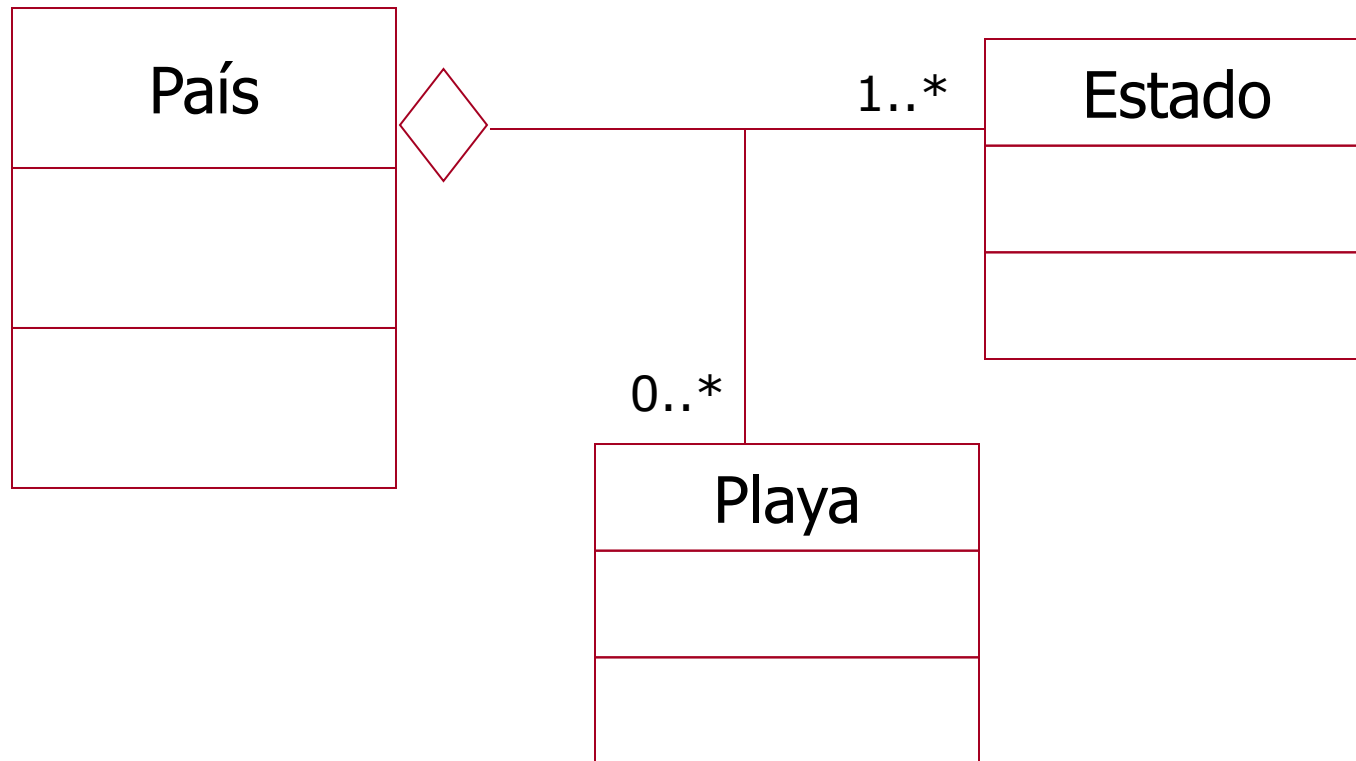
- ◉ Es común que un objeto contenga objetos de otras clases, tal capacidad se llama **agregación**.
- ◉ Por ejemplo: Un objeto Auto *tiene* o *esta formado* por objetos de otras clases (Llantas, Motor, etc.)
- ◉ La relación que se establece entre estos objetos se llama “**tiene un**”, “**es parte de**” o “**consta de**”
 - Ejemplo: Un auto **tiene un** motor
 - Un escuela **tiene** alumnos
- ◉ La relación de agregación, es una relación “débil” entre los objetos, ya que estos pueden existir independientemente del “todo”. Es decir, una llanta “puede existir”, sin ser parte de un auto.

RELACIÓN DE AGREGACIÓN: REPRESENTACIÓN EN UML



Agregación: diamante vacío

RELACIÓN DE AGREGACIÓN: OTRO EJEMPLO

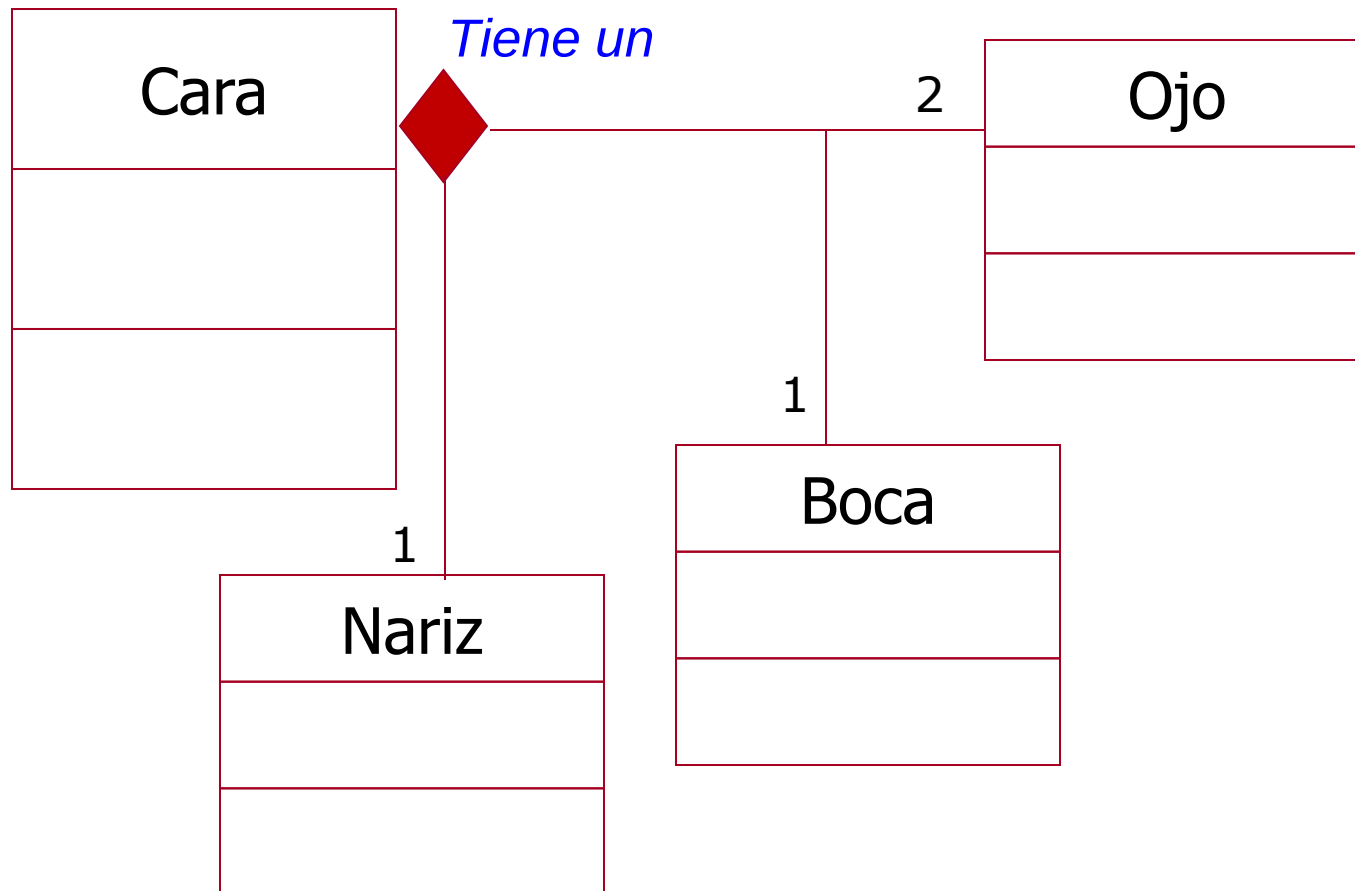


RELACIÓN DE COMPOSICIÓN

- ◉ Es una relación de composición es muy parecida a la anterior (del tipo **tiene un**), sólo que en este caso la relación es **mas fuerte**.
- ◉ La vida de los objetos incluidos como atributos es totalmente dependiente de la vida del objeto contenedor
 - Por ejemplo, una cara puede ser descrita, por los elementos que la forman; es decir; una cara está compuesta por un par de ojos, una boca y una nariz
 - Tanto los ojos, la boca y la nariz, son objetos de otras clases: la clase Ojo, la clase Boca y la clase Nariz; sin embargo no existen independientemente a una cara.

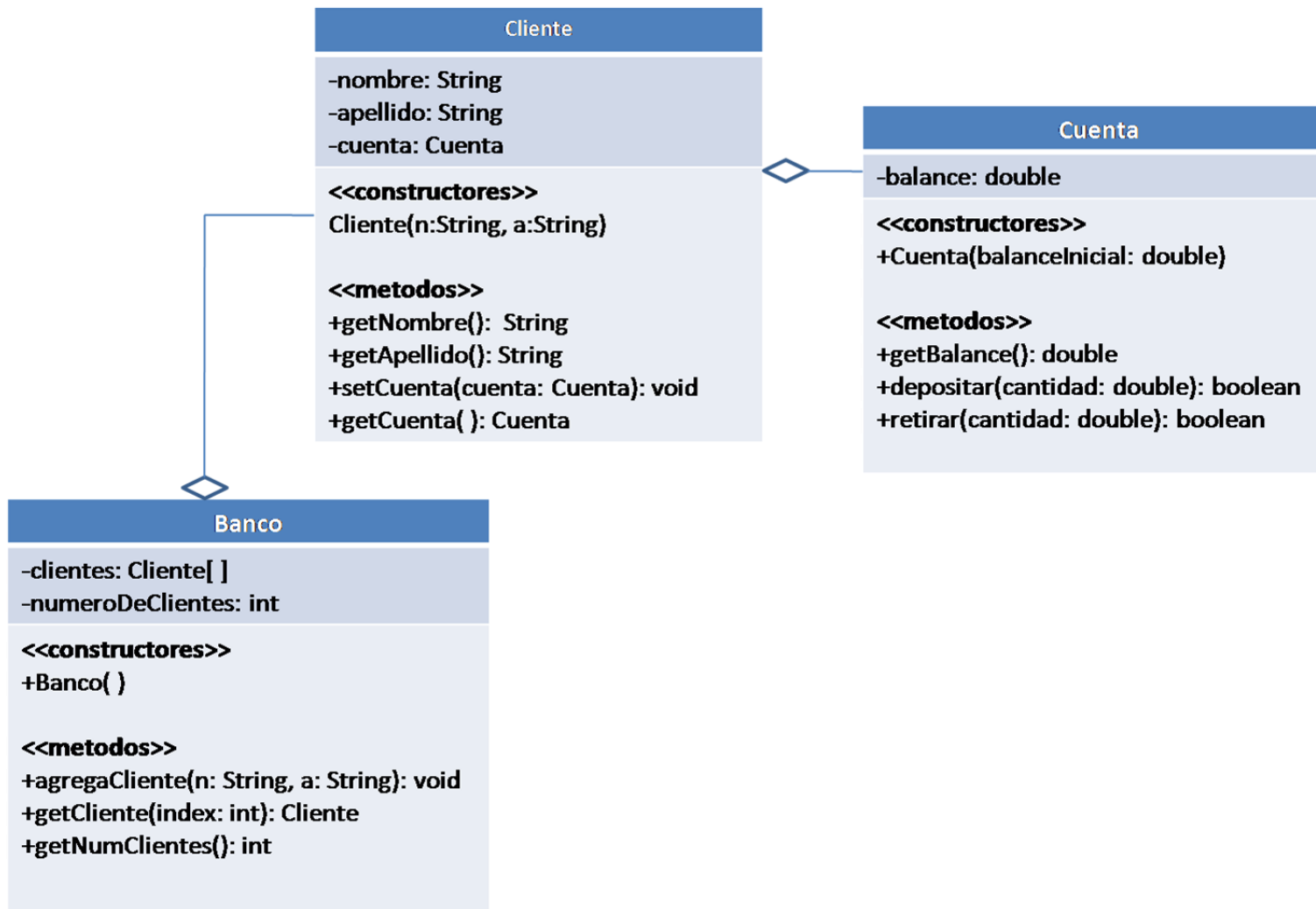


RELACIÓN DE COMPOSICIÓN:UML



Composición: diamante lleno

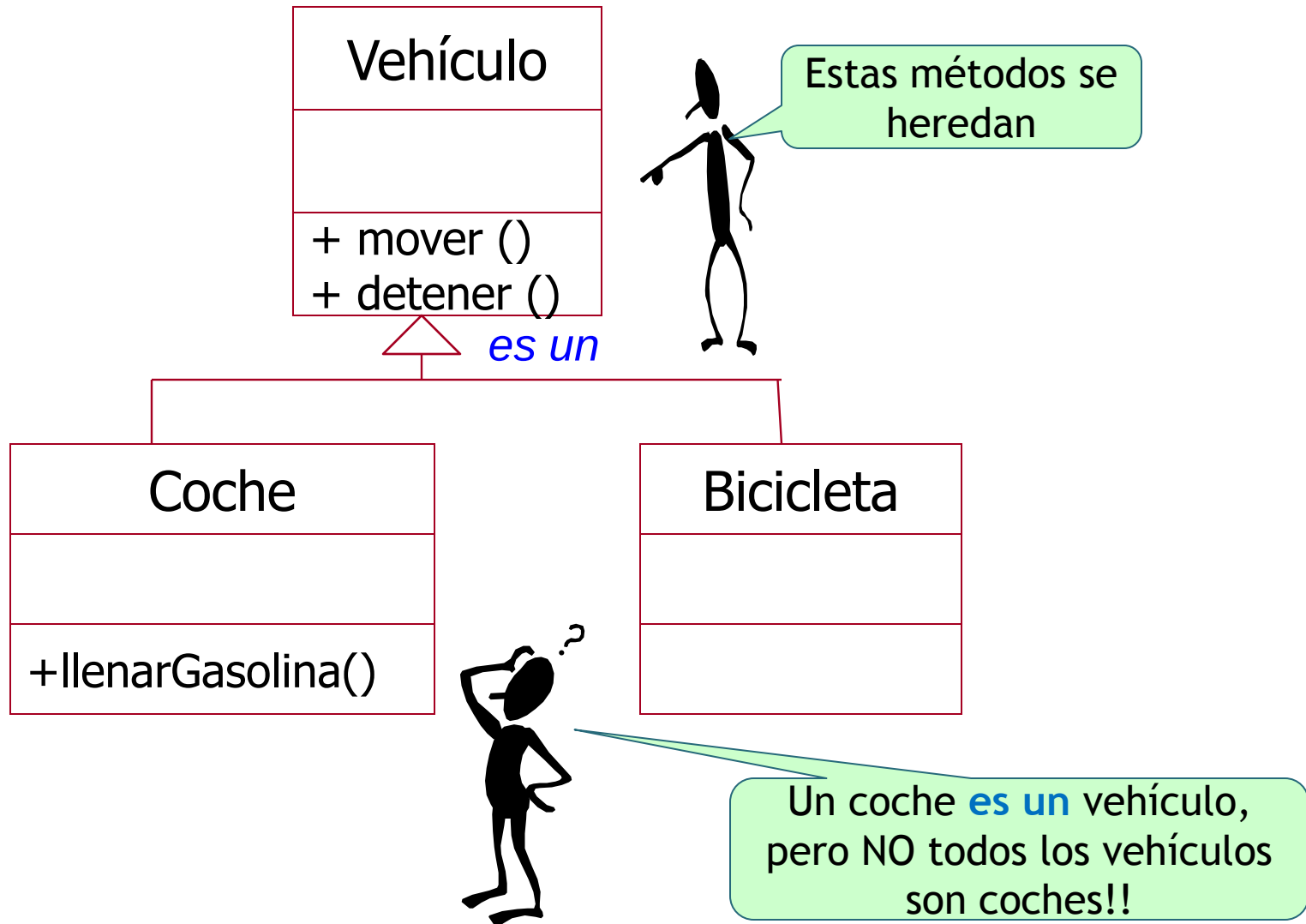
OTRO EJEMPLO



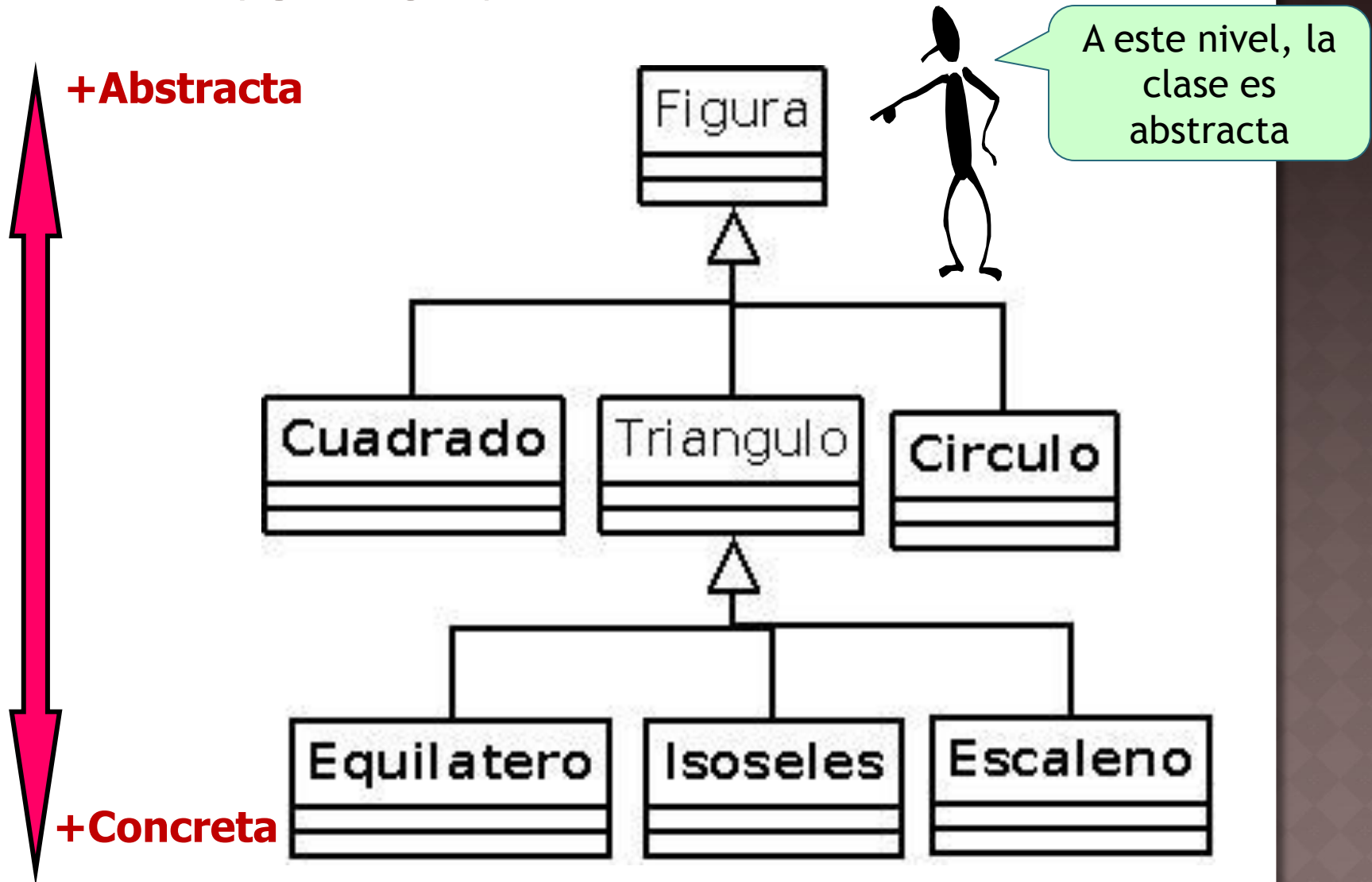
RELACIÓN DE HERENCIA O GENERALIZACIÓN

- ◉ Como ya comentamos, la **herencia** permite especificar jerarquías de clases en las que unas clases toman propiedades de otras (atributos, operaciones y relaciones).
- ◉ A la clase superior se la llama **superclase** mientras que a la clase que hereda las propiedades de la anterior se la llama **subclase**.
 - ▶ La relación que se establece entre clases que heredan, se llama "**es un**"
 - ▶ Por ejemplo:
 - ▶ Una persona **es un** ser humano
 - ▶ Un perro **es un** canino
 - ▶ Un auto **es un** medio de transporte

RELACIÓN DE HERENCIA EN UML

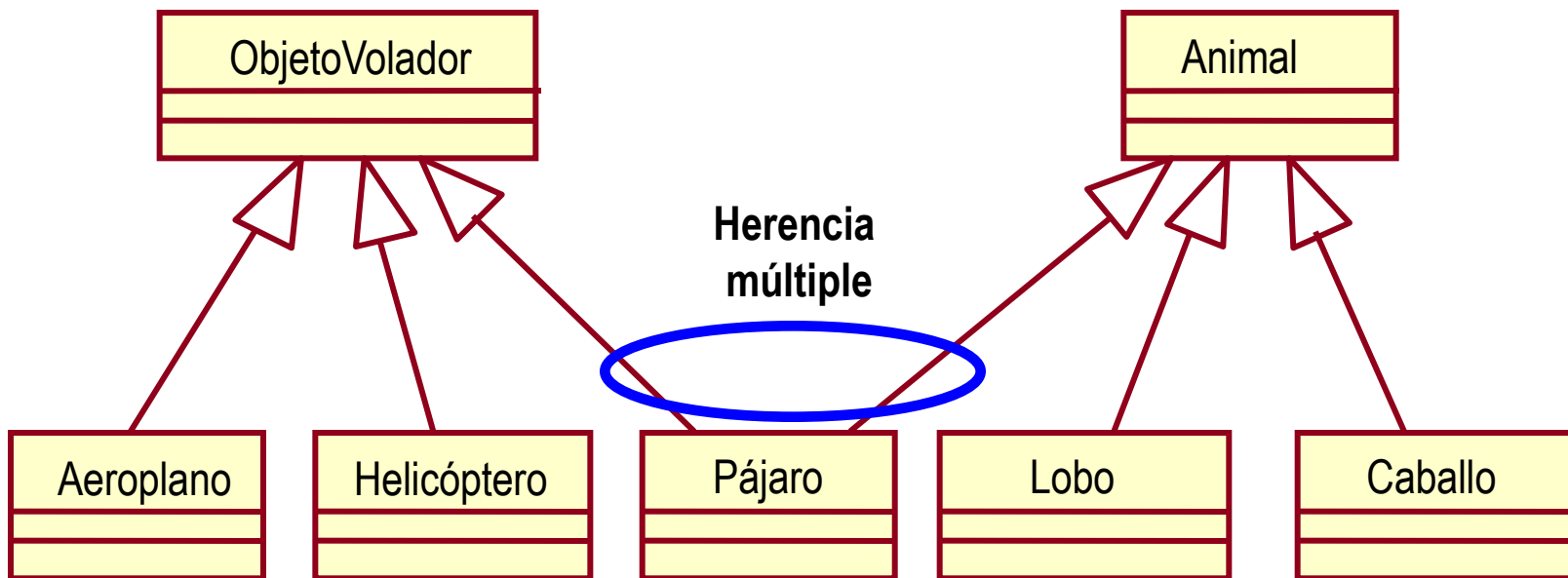


HERENCIA SIMPLE



HERENCIA MÚLTIPLE

- ◉ La mayoría de las veces una clase hereda de solo otra clase (herencia simple), sin embargo una misma clase puede heredar de varias clases distintas (herencia multiple).
- ◉ Esto puede ayudar a simplificar el Modelo de Analisis, pero debe usarse juiciosamente, ya que puede ser dificil de implementar en el Modelo de Diseño.

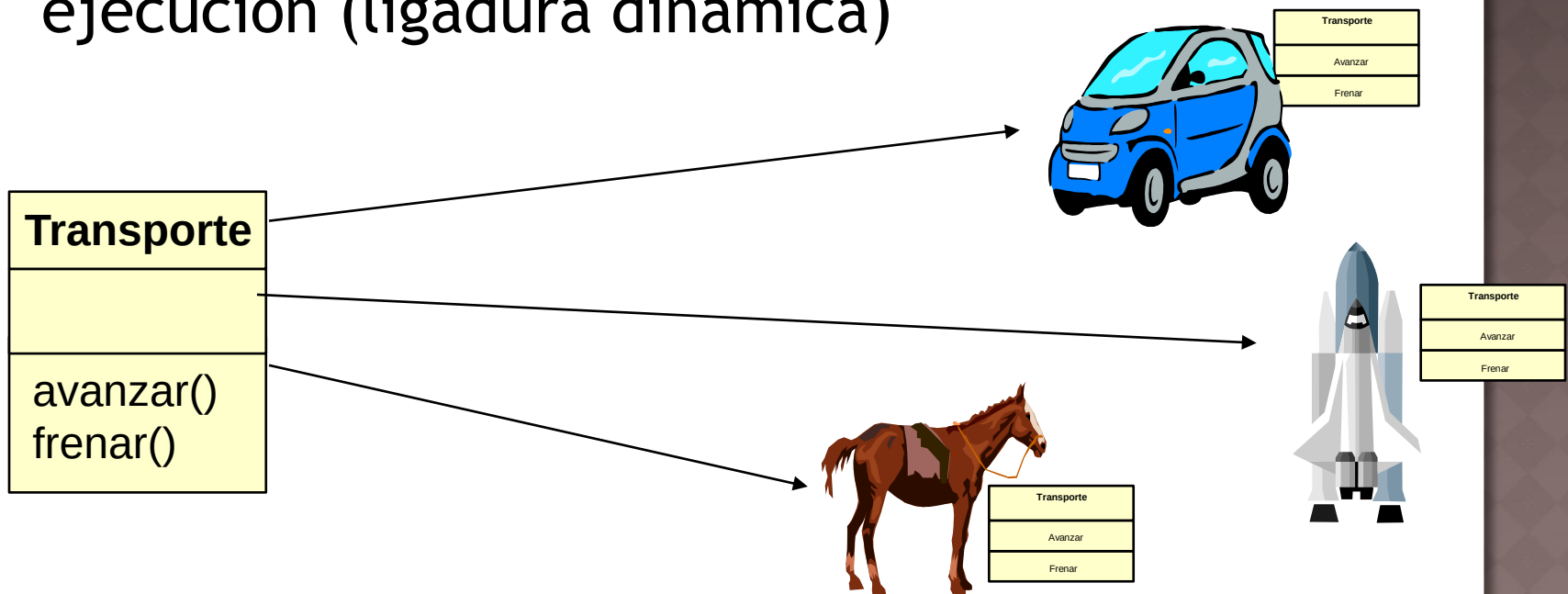


POLIMORFISMO

- ⦿ Recordamos que **polimorfismo** se define como la capacidad de objetos de clases diferentes, relacionados mediante herencia, a responder de **forma distinta** al **mismo mensaje** (llamada a un método)
- ⦿ Es decir, el mismo mensaje que se envía a muchos tipos de objetos, toma “muchas formas” y de ahí viene el término polimorfismo.

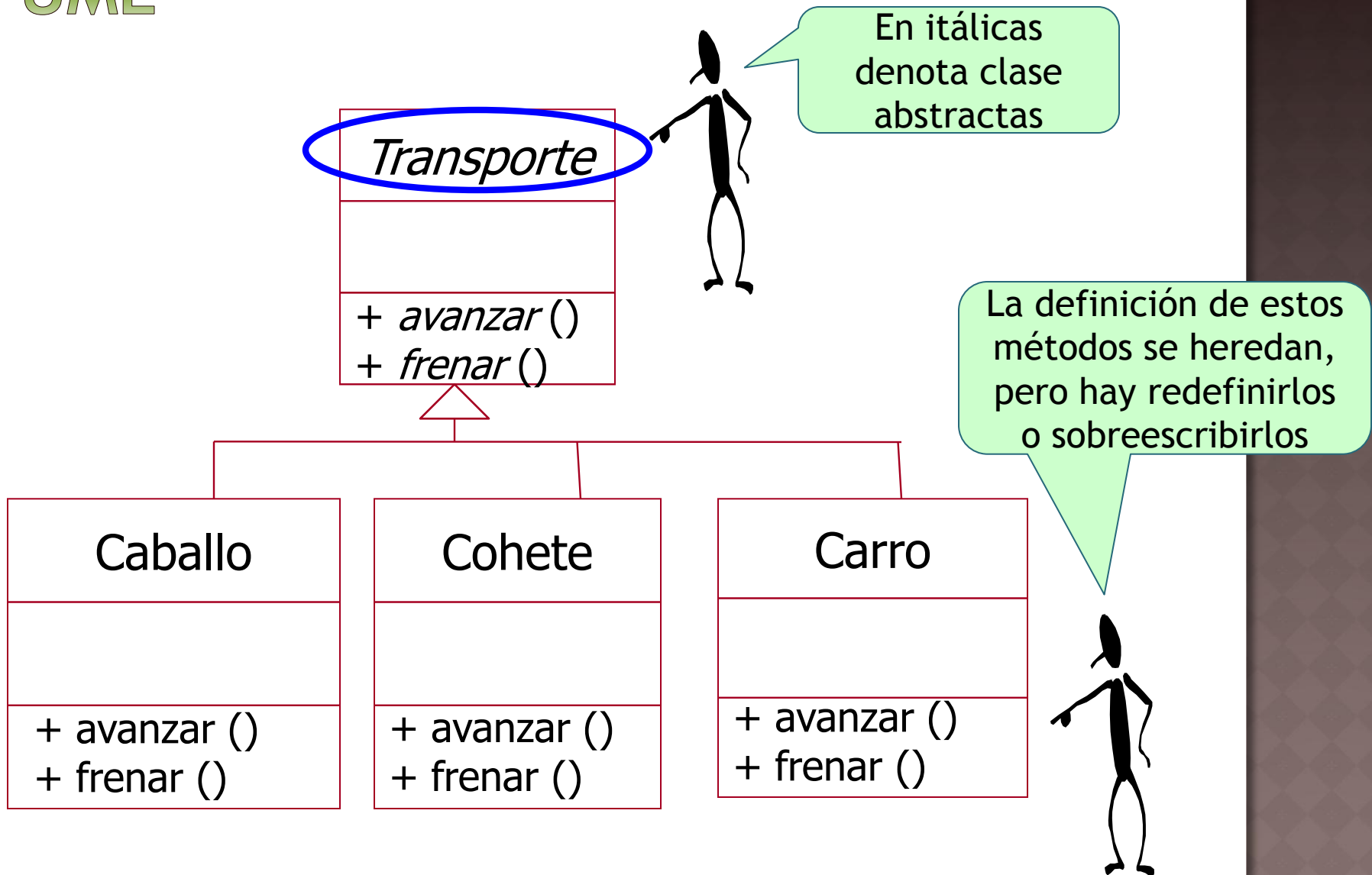
POLIMORFISMO - EJEMPLO

- ◉ La **definición** del método reside en la superclase
- ◉ La **implementación** del método reside en la subclase
- ◉ La invocación es **resuelta** al momento de ejecución (ligadura dinámica)



REPRESENTACIÓN POLIMORFISMO

UML



REVISEMOS LO APRENDIDO

- ✓ Clase
- ✓ Encapsulación
- ✓ Relaciones entre clases
 - ✓ Asociación
 - ✓ Dependencia
 - ✓ Agregación
 - ✓ Composición
 - ✓ Herencia
- ✓ Polimorfismo



CONTENIDO

- ¿Qué es UML?
 - Diagramas UML
- Diagrama de clases
 - Clase
 - Relaciones entre clases
 - Asociación
 - Dependencia
 - Agregación
 - Composición
 - Herencia
 - Realización
- **Diseño OO**

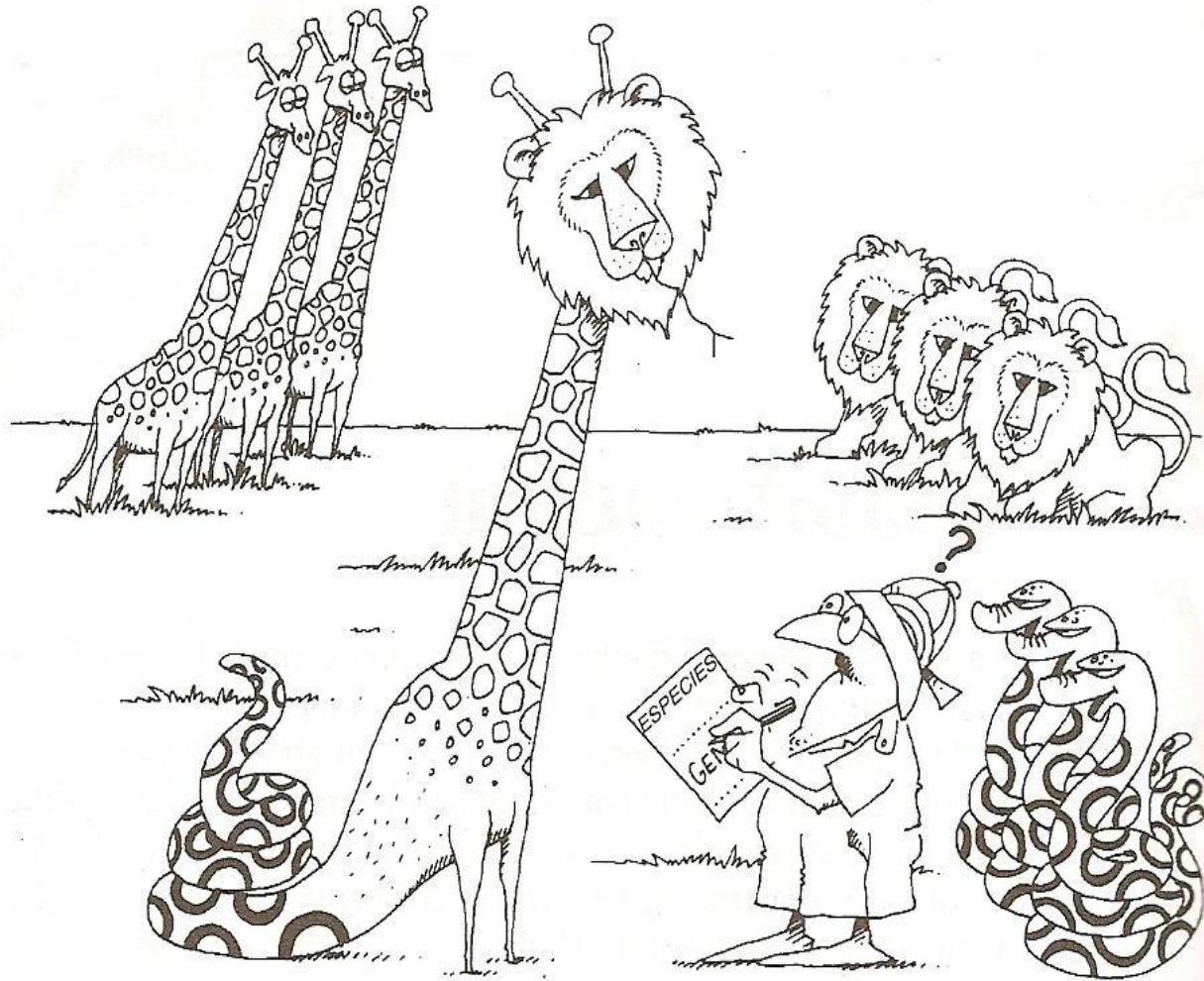


RESOLUCIÓN DE PROBLEMAS USANDO POO

- ◉ Para resolver un problema usando POO se deben seguir los siguientes pasos
 - **Análisis**
 - En esta etapa se deben identificar todos los objetos relacionados en la descripción del problema
 - **Diseño**
 - Clasificar los objetos
 - Obtener sus características y operaciones principales.
 - Identificar las relaciones entre los objetos
 - **Codificación**
 - Escribir el código de cada objeto sin tener en cuenta al resto de elementos del programa
 - Implementar las relaciones

LA IMPORTANCIA DE UNA CLASIFICACIÓN CORRECTA

- ◉ La identificación de clases y objetos es la parte más difícil del diseño orientado a objetos
- ◉ La identificación de objetos implica descubrimiento e invención
- ◉ No hay recetas fáciles para identificar clases y objetos
- ◉ Clasificar es agrupar cosas que tienen una estructura común o exhiben un comportamiento común
- ◉ La clasificación ayuda a identificar jerarquías de generalización, especialización, y agregación entre clases.
- ◉ La clasificación, también proporciona una guía para tomar decisiones sobre modularización



- La clasificación es el medio por el cual ordenamos el conocimiento

UN MÉTODO SENCILLO PARA IDENTIFICAR OBJETOS

- ⦿ Escribe la descripción del problema
 - Identificar la meta
- ⦿ Subraya lo siguiente:
 - **sustantivos**: candidatos a clases
 - **verbos**: candidatos a métodos
 - **adjetivos**: candidatos a atributos

EJEMPLO

- ⦿ Hacer un programa que permita realizar operaciones aritméticas sobre fracciones. Además, permita imprimir el resultado en la forma a/b, donde a es el numerador y b el denominador.

Candidatos a Clases

Candidatos a métodos

Candidatos a atributos

EJEMPLO

- ◉ En el rancho la "Esperanza", se cuenta con diferentes graneros para almacenar semillas (maíz, trigo, sorgo, etc.). Los graneros tienen diferentes formas y tamaños, algunos son cilíndricos, otros cúbicos y otros cónicos. Conocer el volumen de semillas almacenadas es vital para la subsistencia de la región en tiempos de sequías; por lo que se requiere calcular constantemente la cantidad total de semillas almacenadas en los graneros, sin importar el tipo de semilla, para evitar desabasto. Además, visualizar el tipo de semilla que está almacenada en cada granero

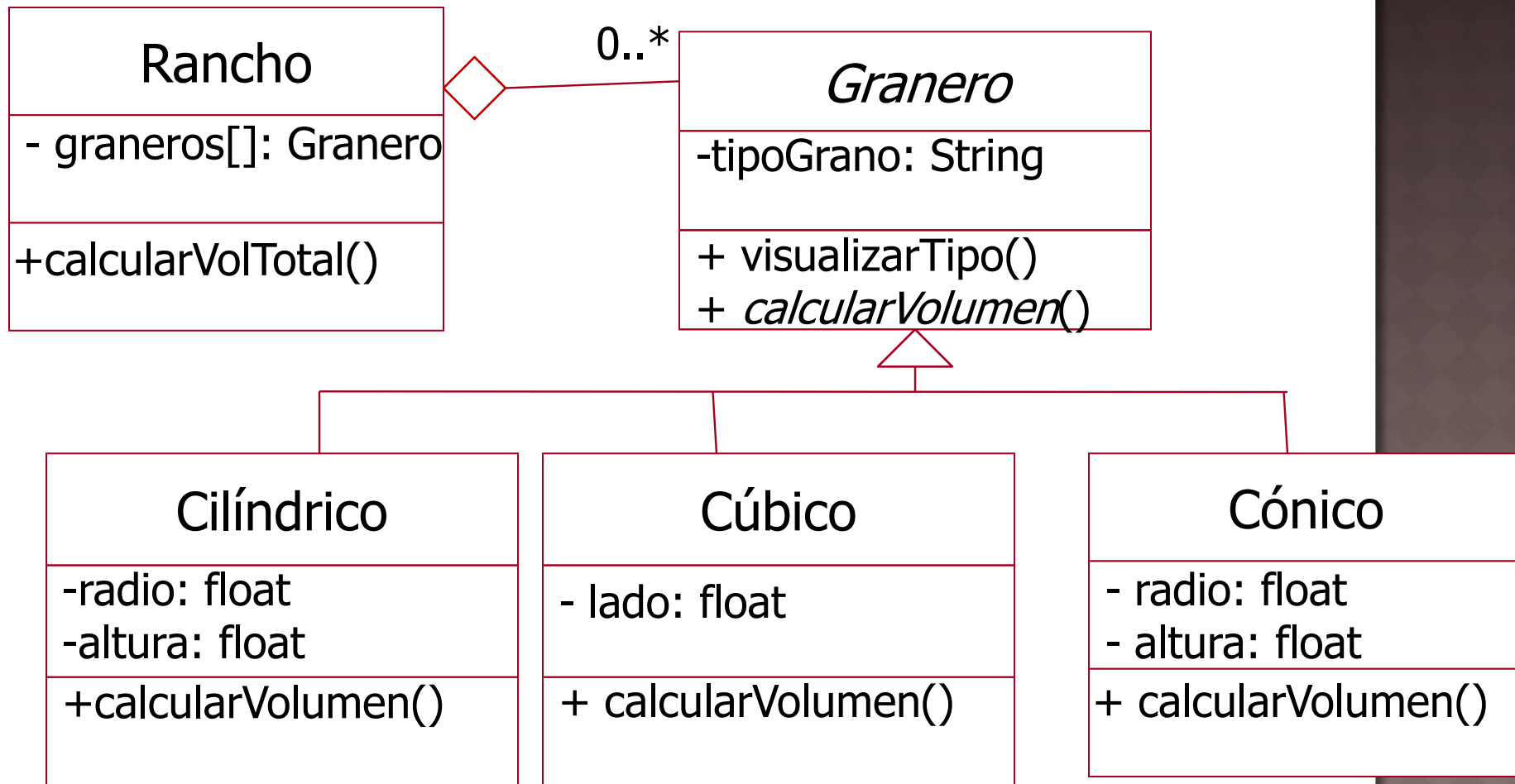
ANALIZAR OBJETOS, ATRIBUTOS, MÉTODOS

Elemento	Aceptado
Rancho	✓
Graneros	✓
Semillas	X
Maiz	X
Sorgo	X
Trigo	X
Región	X
Conocer volumen	✓
Calcular volumen total	✓
Visualizar tipo semillas	✓
Formas	✓ (cilindro, cubo, cónicos)
Tamaño	✓ (medidas)
Tipo de semilla	✓

CONSTRUIR DIAGRAMA DE CLASES

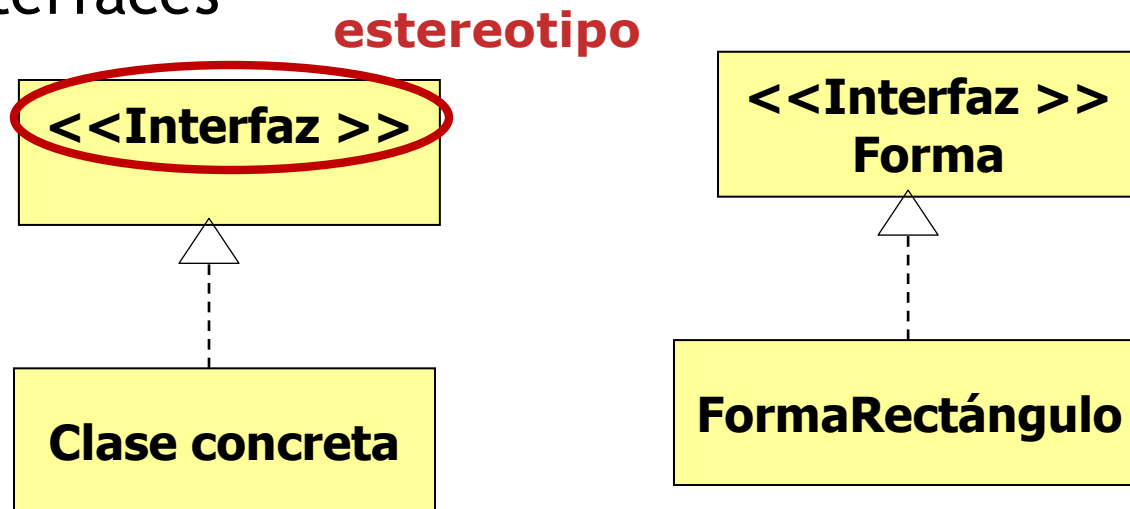
- ⦿ Para cada objeto encontrado, modelar su clase por separado
- ⦿ Relacionar las clases
- ⦿ Fórmulas para cálculo de volúmenes:
 - $\text{cubo} = \text{lado}^3$
 - $\text{cilindro} = \text{PI} * \text{radio}^2 * \text{Altura}$
 - $\text{cono} = (\text{PI} * \text{radio}^2 * \text{Altura}) / 3$

DISEÑO (DIAGRAMA DE CLASES)



RELACIÓN DE REALIZACIÓN

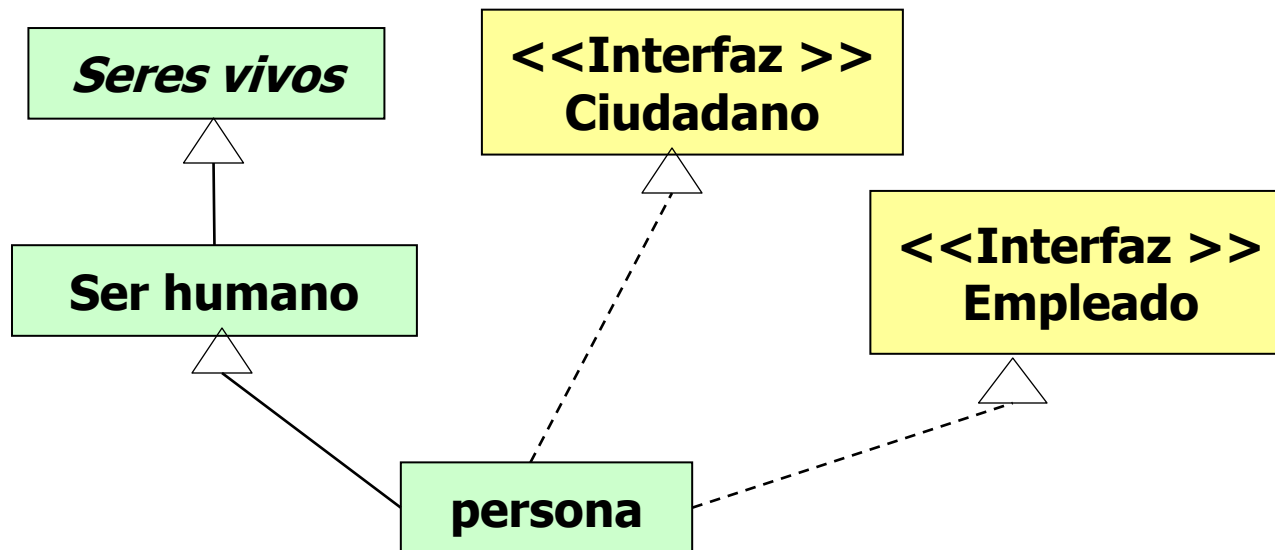
- Una relación de realización, indica que una clase **implementa** un comportamiento específico de otra clase, llamada **interface** o protocolo.
- Una interface puede ser **realizada** por muchas clases
- Una clase concreta puede **implementar** muchas interfaces



INTERFACES

- ⦿ Una **interface** es como una clase abstracta, sólo que todos sus métodos son siempre **abstractos**.
- ⦿ Las interfaces son utilizadas para definir roles representados por objetos.
 - Una misma persona puede jugar varios roles como por ejemplo: maestro, padre, hermano, esposo, etc.
- ⦿ De esta forma, la relación de realización que se establece entre la clase y la interface es también una relación “**es-un**”

HERENCIA MÚLTIPLE



- Una clase puede heredar de **solo una** clase, pero puede **implementar muchas** interfaces

¿QUE HEMOS APRENDIDO?

